

WeaveCube

Exact Solution Construction with the Lee Weave Method

Technical Report and Reproducibility Note

Version 1.0 | 16 September 2026

Lee Sihyun / WeaveCube Project

Scope

Shared-word exact construction, A18 exact signature joins, FIRST_WITNESS late materialization, exact ATO characterization, and sealed operational evidence.

Abstract

WeaveCube is an experimental Rubik's Cube research system centered on the Lee Weave Method (LWM): an exact shared-word construction method that maintains multiple compatible move words, progressively intersects heterogeneous exact constraints over those same words, and delays explicit materialization until the compatible set becomes small. CubeLab remains the historical/internal development name in versioned logs, receipts, and local paths where changing it would damage reproducibility. This report consolidates the strongest currently validated results and explicitly separates mechanism claims from engineering optimizations and from established prior art.

The clearest mechanism result is the A18 5+5 split exact-signature join. Two frozen banks of 577,368 depth-5 words imply 333,353,807,424 conceptual word pairs, yet exact signature intersection avoids runtime Cartesian enumeration until the candidate count falls below a small materialization threshold. In the final ordering ablation, both query-specific marginal ordering and a fixed exact piece order found witnesses in all 8/8 cases; adaptive ordering reduced the median threshold step from 10 to 8 and yielded a modest runtime benefit, supporting the conclusion that progressive exact intersection is the core mechanism while ordering is secondary.

A separate FIRST_WITNESS line showed that exactness does not require materializing the complete compatible family: a same-process comparison reduced representative closure wall from a 36.8624 s geometric-mean full closure to 15.4988 s for the first exact witness, a 2.378x hot speedup, with native replay passing. For the ATO terminal predicate, WeaveCube established an exact finite-set characterization of the physical ATO-zero set as six pairwise-disjoint half-turn orbits of 663,552 states each, totaling 3,981,312 states. Finally, a sealed 24-case deterministic 25-move random-move corpus was solved 24/24 with 24/24 replay verification, median case wall 1.224 s, p90 3.017 s, and maximum 4.914 s.

Executive Summary

Result	Status	Evidence
Shared-word progressive exact restriction	Validated	A18 exact signature join
333.35B conceptual 5+5 pair space without early Cartesian materialization	Validated	A18 5+5
Adaptive ordering as secondary optimization	Validated	v37.530 A/B/A, all arms 8/8
FIRST_WITNESS late materialization	Validated	v37.507 same-process comparison, 2.378x hot speedup
Physical ATO-zero = six disjoint half-turn orbits	Proved	v37.470 finite-set proof
Sealed operational sample	24/24 PASS	v37.527 FULL-24
Universal random-state solve rate	Not claimed	Sample is deterministic random-move, not uniform random states
General superiority over Kociemba/Nissy	Not claimed	No controlled head-to-head benchmark

Contents

1. Research Question and Contribution Boundary
 2. Prior Art and Positioning
 3. Shared-Word Exact Construction
 4. A18 Split Exact-Signature Join
 5. Ordering Ablation
 6. Late Materialization and FIRST_WITNESS
 7. Correctness, Authority, and End-to-End Economics
 8. Exact ATO Characterization
 9. Sealed FULL-24 Operational Benchmark
 10. Limitations and Non-Claims
 11. Reproducibility and Public Release
 12. Conclusion
- Appendix A. Evidence Receipts
- Appendix B. References

1. Research Question and Contribution Boundary

The WeaveCube research question is not simply whether a Rubik's Cube can be solved. Highly effective solvers already exist. The narrower question is whether an exact solution can be constructed by maintaining a shared

language of compatible move words, progressively applying exact heterogeneous constraints to the same words, and postponing commitment and materialization.

1.1 Core contribution candidates

- Shared-word exact construction: all active conditions refer to the same candidate word rather than independently chosen per-piece words.
- Progressive exact restriction: exact conditions shrink a compatible language without granting heuristic scores semantic authority.
- Late materialization / FIRST_WITNESS: explicit pair/family construction is delayed until small, and a first exact witness is sufficient for SAT authority.
- Exact physical ATO characterization: the project-specific ATO-zero predicate is exactly the union of six pairwise-disjoint half-turn orbits.

1.2 Claims explicitly excluded

- WeaveCube does not claim to have invented subgroup solving, cosets, IDA*, pruning tables, SAT encodings, decision diagrams, generic set intersection, or generic product automata.
- WeaveCube does not claim a universal random-state solve rate from the FULL-24 sample.
- WeaveCube does not claim general performance superiority over Kociemba, Nissy, or other mature cube solvers.
- The A18 planted-positive mechanism validation is not a theorem that the same construction succeeds for every longer root.

2. Prior Art and Positioning

The novelty boundary is intentionally conservative. Kociemba's two-phase method uses subgroup structure and heuristic search; the first phase drives a cube into $G1 = \langle U, D, R2, L2, F2, B2 \rangle$, after which the second phase solves inside that subgroup [1]. The 20-move diameter proof also relies heavily on subgroup/coset decomposition and large-scale exact computation [2].

Minato's piDD represents sets of permutations compactly and supports algebraic set operations including union, intersection, and a Cartesian product; Rubik's Cube analysis is explicitly included as an application [3]. Maehara and Inoue's Group Decision Diagram likewise supports membership and set operations over permutation sets, including Cartesian product [4]. Chen demonstrated a Rubik's Cube SAT encoding in 2011 [5]. Accordingly, WeaveCube's contribution should not be phrased as the invention of set intersection, product constructions, SAT constraints, or coset methods.

The differentiating focus in WeaveCube is architectural: preserve multiple exact compatible witnesses over one shared move-word language, progressively intersect heterogeneous exact constraints, and materialize only when the exact compatible set has collapsed enough to make explicit construction economical.

3. Shared-Word Exact Construction

WeaveCube's central invariant is that heterogeneous conditions must remain synchronized on the same move word. This matters because a collection of individually satisfiable per-piece conditions does not imply the existence of one shared move sequence satisfying all of them. The system therefore treats the compatible word set itself as the object being refined.

Stage	Meaning
Start with a finite exact word language	A word bank or exact symbolic language represents candidate move sequences.
Apply an exact condition	A piece-Q, ATO, boundary, or other admissible exact condition removes incompatible words.
Preserve all surviving compatible witnesses	No greedy commitment is required at this stage.
Repeat	Additional exact dimensions continue to restrict the same shared-word set.
Materialize only when small	Explicit pairs or a first witness are constructed only after exact counts cross a bounded threshold.
Replay in native full-Q semantics	The final witness is checked by native replay; this is the final correctness authority.

4. A18 Split Exact-Signature Join

A18 provides the clearest mechanism demonstration. The frozen depth-5 left and right word banks each contain 577,368 words. The conceptual Cartesian product therefore contains 333,353,807,424 pairs. The implementation does not enumerate those pairs up front. Instead it accumulates exact packed signature dimensions and computes the exact compatible pair count after each added condition.

Figure 1. Representative A18 5+5 case. The conceptual pair space is reduced by exact conditions from 333.35B to 19 before explicit pair materialization.

For the representative case shown above, the compatible counts were 333,353,807,424 -> 7,805,234,752 -> 295,074,147 -> 11,530,124 -> 613,381 -> 55,543 -> 3,021 -> 326 -> 19. The important statement is not that 333 billion pairs were searched. They were represented implicitly through exact signature counts, and runtime pair enumeration remained zero until the candidate threshold.

4.1 Mechanism properties

- Frozen long-word tables are not constructed.
- Runtime move-tree DFS/BFS is not used by this join mechanism.
- The join state is exact 20-piece Q.
- Condition joins use packed exact signature equality.
- The first exact witness terminates the query.
- No global fixed solution horizon is introduced by the mechanism.

5. Ordering Ablation

To separate the effect of progressive exact intersection from the effect of condition ordering, the final v37.530 ablation compared query-specific exact marginal ordering against a fixed piece order 0..19. All other semantics were held constant: same 5+5 cases, same seed, same exact signatures, same candidate cap (64), same maximum number of conditions (12), and the same replay semantics.

Figure 2. Median exact conditions required to reach the ≤ 64 materialization threshold.

Metric	Adaptive A	Fixed	Adaptive C
--------	------------	-------	------------

Witnesses	8/8	8/8	8/8
Median conditions	8	10	8
Median step to ≤ 64	8	10	8
Median query wall (s)	1.747	1.971	1.789

The fixed arm still found witnesses in all 8/8 cases. Therefore the combinatorial collapse is not created by adaptive ordering. Adaptive ordering is a secondary execution optimization: it reduced the median threshold step from 10 to 8 and produced an approximately 10% median runtime benefit relative to the fixed order. This distinction is important for the contribution claim.

6. Late Materialization and FIRST_WITNESS

The FIRST_WITNESS line tested a stronger form of late materialization. Instead of constructing the full compatible closure language, the system reuses an exact packed product engine and stops at the first exact witness. A no-witness result from this bounded backend is not UNSAT authority; resource exhaustion or no witness falls back to the existing exact path.

Figure 3. Same-process representative closure economics: full frozen closure geometric mean 36.8624 s versus FIRST_WITNESS 15.4988 s.

The representative same-process comparison used full frozen controls at 36.9212 s and 36.8037 s, for a 36.8624 s geometric mean. Vectorized FIRST_WITNESS took 15.4988 s, corresponding to a 2.378x hot speedup. The exact witness matched the full-closure sample, and native replay passed. The packed exact product traversed 1,954,520 states and 1,958,916 edges.

6.1 Authority contract

- FIRST_WITNESS is SAT-only authority.
- Native full-Q replay is the final correctness authority.
- No-witness or resource cap does not become UNSAT; it triggers fallback.
- Local exact EMPTY cannot prune a whole residual-graph child without explicit exhaustive-child authority.

7. Correctness, Authority, and End-to-End Economics

WeaveCube deliberately separates correctness from production economics. A locally faster exact kernel is not automatically promoted if total solve wall is worse. At v37.516, the optimized inline FIRST_WITNESS path was correct and produced graph SAT, but the stable end-to-end comparison still lost: historical controls were 18.638914 s and 18.288457 s (geometric mean 18.462854 s), while policy total was 19.034902 s, a 0.572048 s deficit.

v37.516 end-to-end item	Wall / result
Historical A	18.638914 s, UNKNOWN_RESOURCE_CAP
Optimized inline FIRST_WITNESS	19.034902 s, graph SAT
Historical B	18.288457 s, UNKNOWN_RESOURCE_CAP
Historical geometric mean	18.462854 s
Policy deficit	+0.572048 s

Profiling identified repeated exact common_mask computation as the dominant remaining cost: 6.288846 s in the diagnostic profile, including 4.529536 s at depth 6. The next branch compiled exact per-piece/node/Q move-viability masks so runtime common-mask evaluation could become lookup-and-AND while preserving

the exact graph. The public release should include the final v37.517 revalidation receipt before making a formal end-to-end production-speed claim. The current project policy remains bounded/registered rather than global automatic activation.

8. Exact ATO Characterization

A separate line studied the physical ATO-zero endpoint predicate. The initial conjecture that physical ATO-zero was a single half-turn orbit was false. The completed finite-set proof established six pairwise-disjoint half-turn orbits, each containing 663,552 states, for a total of 3,981,312 physical ATO-zero states.

Figure 4. The physical ATO-zero set is exactly covered by six pairwise-disjoint half-turn orbits.

The proof has a finite-set sandwich structure. First, six actual physical representatives are obtained, one for each delta class. Closing each representative under the six half turns yields 663,552 states. The six closures are pairwise disjoint, so their constructive union gives a lower bound of 3,981,312 physical ATO-zero states. Independently, standard physical invariants yield the same 3,981,312 upper bound. Equality of lower and upper bounds proves exact coverage.

8.1 Terminology

Project artifacts sometimes use left/right coset language under different action conventions. Because the half-turn subgroup is not being treated as normal, this public report uses the action-neutral phrase 'six disjoint half-turn orbits' in the main text. This avoids unnecessary ambiguity while preserving the exact finite-set result.

9. Sealed FULL-24 Operational Benchmark

The final practical benchmark used a predeclared deterministic corpus of 24 random-move scrambles, each 25 moves long. The corpus was sealed before solver import. Its SHA-256 digest was 77bd96782994e5ae41e98ebab013de42354178bb9589e043dc12c4e416fee88e. The benchmark runner later verified that the driver requested exactly 24 cases of length 25 and that the report matched the sealed corpus.

Figure 5. Per-case wall times for the sealed FULL-24 operational corpus. Dashed line: median.

Metric	Result
Solved	24/24
Native/witness replay	24/24 PASS
Fallback solved	0
Resource inconclusive	0
Replay failures	0
Median case wall	1.224 s
p90 case wall	3.017 s
Maximum case wall	4.914 s
Total case wall	39.522 s
Wrapper wall	42.044 s
Solution length	18-23 moves; median 21

All 24 cases solved at the first tested +3 offset. This is strong operational evidence for the frozen configuration on the chosen sample. It is not a uniform random sample over legal cubie states, and it is too small to support a universal solve-rate claim.

10. Limitations and Non-Claims

- The A18 cases are a mechanism validation set and include planted-positive construction assumptions; success is not generalized to all roots.
- The FULL-24 sample is deterministic random-move, not uniform random legal cube states.
- No controlled head-to-head benchmark in this report establishes that WeaveCube is faster than Kociemba, Nissy, or other mature solvers.
- The current C4 production scope is bounded/registered; broad automatic generalization remains unproved.
- Historical 1000-case coverage may be cited as historical supporting evidence only, not as a fresh current benchmark.
- Many performance results depend on frozen caches and exact assets. Public reproducibility requires documenting or regenerating those assets.

These limits are part of the research result. The strongest defensible contribution is an exact construction paradigm and associated finite-set/operational evidence, not a claim that WeaveCube replaces every existing Rubik's Cube solving approach.

11. Reproducibility and Public Release

11.1 Target environment

Item	Target / recorded value
Operating system	Windows
Python	3.14.x in the final presentation/repro runs
Primary project root	C:\python\CubeLab
D6 cache SHA-256	5b985def30317e416a03eb8bde4c039bc7ea1c4d7c34683dffa4ff8cc24055dd0
FULL-24 corpus SHA-256	77bd96782994e5ae41e98ebab013de42354178bb9589e043dc12c4e416fee88e

11.2 Public repository layout

The accompanying publication package is arranged so that documentation and small reproducibility scripts can be public without accidentally committing large caches, working reports, or local machine artifacts. The full WeaveCube solver source should be merged from the current working tree before publication.

Path	Purpose
README.md	Public project overview and claims
docs/WeaveCube_Technical_Report_v1.pdf	Detailed report
presentation/WeaveCube_Core_Deck_v1.pptx	Presentation deck
scripts/repro/	Sealing, tracing, FULL-24, and A18 presentation/repro scripts currently available
evidence/	Public evidence summaries and small logs/receipts
REPRODUCIBILITY.md	Commands, hashes, and validation gates
CLAIMS_AND_EVIDENCE.md	Claim/evidence/non-claim matrix

REFERENCES.bib	Prior-art references
.gitignore	Excludes large caches/reports/local files

11.3 Release gates before making the repository public

- Build a sanitized, history-trimmed public snapshot from the approved source manifest; do not publish the full research tree or its private history.
- Apply Apache-2.0 only after provenance/release-scope closure and final notice review.
- Exclude direct RubikTwoPhase and Nissy teacher/source-semantic integration from the public snapshot.
- Remove secrets, unnecessary local paths, temporary caches, and generated large arrays; document frozen assets by redistribution, regeneration, or digest.
- Run compileall and the real import smoke on the exact sanitized public checkout.
- Freeze the final report/deck/evidence hashes after all wording and scope edits.
- Tag the first public release only after final snapshot and metadata review.

12. Conclusion

WeaveCube's strongest current result is not a new generic set operation or a replacement for established two-phase/SAT/decision-diagram techniques. It is an exact construction architecture that keeps multiple compatible shared move words alive, progressively intersects heterogeneous exact constraints, and delays explicit materialization until the compatible set is small. A18 demonstrates large implicit combinatorial collapse without early Cartesian enumeration; the ordering ablation shows the collapse survives a fixed condition order. FIRST_WITNESS demonstrates that exact SAT construction can stop at one verified witness rather than materializing the full compatible family. The ATO line contributes an exact six-orbit finite-set characterization, and FULL-24 provides a fresh sealed operational sample.

WeaveCube is therefore best described as a two-phase-family solver with a project-specific front-end exact construction mechanism over shared move words; Phase-2-style subgroup completion remains established prior art. The next work is publication engineering - a sanitized history-trimmed release, evidence packaging, and independent reproduction - rather than opening new search branches.

Appendix A. Evidence Receipts

ID	Version / experiment	Key evidence
E1	v37.530 A18 ordering ablation	Adaptive A 8/8, Fixed 8/8, Adaptive C 8/8; median conditions 8/10/8; median walls 1.747/1.971/1.789 s
E2	v37.507 FIRST_WITNESS A/B/A	Full GM 36.8624 s; FIRST_WITNESS 15.4988 s; 2.378x; native replay PASS
E3	v37.516 C4 revalidation	Historical GM 18.462854 s; optimized total 19.034902 s; correctness PASS but economics -0.572048 s
E4	v37.470 ATO six-orbit theorem	6 x 663,552 = 3,981,312; pairwise disjoint; finite-set lower=upper proof
E5	v37.527 FULL-24	24/24 solved; 24/24 replay; median 1.224 s; p90 3.017 s; max 4.914 s

Public-release note: E1 and E5 are directly represented by the presentation/repro scripts and logs in the current workspace. E2-E4 should be accompanied by the corresponding final receipts from the full WeaveCube working tree when the GitHub repository is published.

Appendix B. References

- [1] Herbert Kociemba, The Two-Phase Algorithm / Two-Phase Algorithm Details. [link](#)
- [2] Tomas Rokicki, Herbert Kociemba, Morley Davidson, and John Dethridge, "The Diameter of the Rubik's Cube Group Is Twenty," SIAM Journal on Discrete Mathematics 27(2), 1082-1105, 2013. DOI: 10.1137/120867366. [link](#)
- [3] Shin-ichi Minato, "piDD: A New Decision Diagram for Efficient Problem Solving in Permutation Space," SAT 2011, LNCS 6695, pp. 90-104. DOI: 10.1007/978-3-642-21581-0_9. [link](#)
- [4] Takanori Maehara and Yuma Inoue, "Group Decision Diagram (GDD): A Compact Representation for Permutations," AAAI-19, pp. 2986-2994. [link](#)
- [5] Jingchao Chen, "Solving Rubik's Cube Using SAT Solvers," CoRR abs/1105.1436, 2011. [link](#)